```
import torch, math
from transformers import GPT2LMHeadModel, GPT2Tokenizer, Trainer, TrainingArguments, DataCollatorForLanguageModel
from datasets import Dataset

set_seed(42)
```

Component–I: Fine-Tune GPT-2 as a Product Review Generator (E-Commerce)

```
tokenizer = GPT2Tokenizer.from_pretrained('gpt2')
model = GPT2LMHeadModel.from_pretrained('gpt2')

tokenizer.pad_token = tokenizer.eos_token
model.config.pad_token_id = tokenizer.eos_token_id

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limi
tokenizer_config.json:   0%|          | 0.00/26.0 [00:00<?, ?B/s]
vocab.json: 0.00B [00:00, ?B/s]
merges.txt: 0.00B [00:00, ?B/s]
tokenizer.json: 0.00B [00:00, ?B/s]
config.json:   0%|          | 0.00/665 [00:00<?, ?B/s]
model.safetensors:   0%|          | 0.00/548M [00:00<?, ?B/s]
Loading weights:   0%|          | 0/148 [00:00<?, ?it/s]
GPT2LMHeadModel LOAD REPORT from: gpt2
Key                    | Status     |  |
-----------------------+------------+--+-
h.{0...11}.attn.bias | UNEXPECTED |  |

Notes:
- UNEXPECTED   :can be ignored when loading from different task/architecture; not ok if you expect identical arch
generation_config.json:   0%|          | 0.00/124 [00:00<?, ?B/s]
GPT2LMHeadModel(
  (transformer): GPT2Model(
    (wte): Embedding(50257, 768)
    (wpe): Embedding(1024, 768)
    (drop): Dropout(p=0.1, inplace=False)
    (h): ModuleList(
      (0-11): 12 x GPT2Block(
        (ln_1): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
        (attn): GPT2Attention(
          (c_attn): Conv1D(nf=2304, nx=768)
          (c_proj): Conv1D(nf=768, nx=768)
          (attn_dropout): Dropout(p=0.1, inplace=False)
          (resid_dropout): Dropout(p=0.1, inplace=False)
        )
        (ln_2): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
        (mlp): GPT2MLP(
          (c_fc): Conv1D(nf=3072, nx=768)
          (c_proj): Conv1D(nf=768, nx=3072)
          (act): NewGELUActivation()
          (dropout): Dropout(p=0.1, inplace=False)
        )
      )
    )
    (ln_f): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
  )
  (lm_head): Linear(in_features=768, out_features=50257, bias=False)
)
```

```
def generate_text(model, tokenizer, prompt, max_length=100):
    model.eval()
    inputs = tokenizer.encode(prompt, return_tensors='pt').to(device)

    with torch.no_grad():
        outputs = model.generate(
            inputs,
            max_length=max_length,
            temperature=0.8,
            top_k=50,
            do_sample=True,
            pad_token_id=tokenizer.eos_token_id
        )

    return tokenizer.decode(outputs[0], skip_special_tokens=True)
```

```
review_prompts = [
    'This product is',
    'I bought this phone and',
    'The quality of this item'
]

print("=== BASELINE REVIEWS ===")

baseline = {}
for p in review_prompts:
    baseline[p] = generate_text(model, tokenizer, p)
    print(f"\nPrompt: {p}")
    print(f"Output: {baseline[p]}")
```

```
The attention mask is not set and cannot be inferred from input because pad token is same as eos token. As a cons
=== BASELINE REVIEWS ===

Prompt: This product is
Output: This product is designed to be used in conjunction with any other device and is intended for use in conju

The following devices may be inserted into any such device, even if the device does not have a user interface or

a) any personal computer;

b) other personal computer or computer-based mobile device;

c) any computer-based home computer; and

d) any personal computer-based network-

Prompt: I bought this phone and
Output: I bought this phone and started working on it. I'm quite pleased with the way it performs, but this phone

Prompt: The quality of this item
Output: The quality of this item is the same as that of any other item on the item page.
```

```
corpus = [
    'this phone has an amazing battery life and the camera quality is outstanding for the price.',
    'i bought this laptop for college and it handles all my assignments and coding projects perfectly.',
    'the sound quality of these headphones is incredible with deep bass and clear vocals.',
    'this smartwatch tracks my steps accurately and the heart rate monitor is very reliable.',
    'great wireless earbuds with noise cancellation that blocks out all background sound.',
    'the keyboard feels very comfortable for long typing sessions and the backlight is a nice touch.',
    'this portable charger saved me during travel and it charges my phone three times on a single charge.',
    'the tablet screen is bright and colorful which makes watching movies a great experience.',
    'i love this fitness tracker because it motivates me to reach my daily exercise goals.',
    'this bluetooth speaker is compact but delivers surprisingly loud and clear audio.',
    'the delivery was fast and the product was packed securely with no damage at all.',
    'excellent value for money and the build quality feels premium despite the affordable price.',
    'the customer service team was very helpful when i had questions about the product features.',
    'this camera takes stunning photos in low light and the video recording quality is very smooth.',
    'i have been using this product for three months and it still works perfectly like day one.',
    'the design is sleek and modern and it looks great on my desk next to my other gadgets.',
    'easy to set up right out of the box and the instructions were clear and simple to follow.',
    'highly recommend this product to anyone looking for quality and reliability at a fair price.',
    'the software updates keep adding new features which makes this purchase even more worthwhile.',
    'best purchase i made this year and i would definitely buy from this brand again.'
]

dataset = Dataset.from_dict({'text': corpus})
```

```
tokenized = dataset.map(
    lambda x: tokenizer(x['text'], truncation=True, padding='max_length', max_length=128),
    batched=True,
    remove_columns=['text']
)

split = tokenized.train_test_split(test_size=0.15, seed=42)
```

```
Map:   0%|          | 0/20 [00:00<?, ? examples/s]
```

```
training_args = TrainingArguments(
    output_dir='./gpt2-reviews',
    num_train_epochs=10,
    per_device_train_batch_size=4,
    learning_rate=5e-5,
    weight_decay=0.01,
    logging_steps=10,
    fp16=torch.cuda.is_available()
)
```

```
trainer = Trainer(
```

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=split['train'],
    eval_dataset=split['test'],
    data_collator=data_collator
)

trainer.train()
```

```
`loss_type=None` was set in the config but it is unrecognized. Using the default loss: `ForCausalLMLoss`.
```

[30/30 00:07, Epoch 10/10]

Step	Training Loss
10	3.779194
20	2.235526
30	2.138841

```
Writing model shards:   0%|          | 0/1 [00:00<?, ?it/s]
TrainOutput(global_step=30, training_loss=2.7178539911905926, metrics={'train_runtime': 9.4655,
'train_samples_per_second': 17.96, 'train_steps_per_second': 3.169, 'total_flos': 11104911360000.0,
'train_loss': 2.7178539911905926, 'epoch': 10.0})
```

```
eval_res = trainer.evaluate()
print("Perplexity:", math.exp(eval_res["eval_loss"]))

print("\n=== FINE-TUNED REVIEWS ===")

for p in review_prompts:
    ft = generate_text(model, tokenizer, p)
    print(f"\nPrompt: {p}")
    print(f"Baseline:   {baseline[p][:120]}")
    print(f"Fine-Tuned: {ft[:120]}")
```

[1/1 : < :]

```
Perplexity: 15.497885306265214

=== FINE-TUNED REVIEWS ===

Prompt: This product is
Baseline:   This product is designed to be used in conjunction with any other device and is intended for use in c
Fine-Tuned: This product is made using recycled recycled plastic and the product comes with a sturdy and affordab

Prompt: I bought this phone and
Baseline:   I bought this phone and started working on it. I'm quite pleased with the way it performs, but this p
Fine-Tuned: I bought this phone and it is fast and reliable for the money. I would definitely purchase from this

Prompt: The quality of this item
Baseline:   The quality of this item is the same as that of any other item on the item page.
Fine-Tuned: The quality of this item is outstanding and the quality control is very helpful. The product features
```

Component–II: Fine-Tune GPT-2 as a Recipe Instruction Generator (Food-Tech)

```
tokenizer2 = GPT2Tokenizer.from_pretrained('gpt2')
model2 = GPT2LMHeadModel.from_pretrained('gpt2')

tokenizer2.pad_token = tokenizer2.eos_token
model2.config.pad_token_id = tokenizer2.eos_token_id

model2.to(device)
```

```
Loading weights:   0%|          | 0/148 [00:00<?, ?it/s]
GPT2LMHeadModel LOAD REPORT from: gpt2
Key                  | Status     |  |
---------------------+------------+--+-
h.{0...11}.attn.bias | UNEXPECTED |  |

Notes:
- UNEXPECTED   :can be ignored when loading from different task/architecture; not ok if you expect identical arch
GPT2LMHeadModel(
  (transformer): GPT2Model(
    (wte): Embedding(50257, 768)
    (wpe): Embedding(1024, 768)
    (drop): Dropout(p=0.1, inplace=False)
    (h): ModuleList(
      (0-11): 12 x GPT2Block(
        (ln_1): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
        (attn): GPT2Attention(
          (c_attn): Conv1D(nf=2304, nx=768)
          (c_proj): Conv1D(nf=768, nx=768)
          (attn_dropout): Dropout(p=0.1, inplace=False)
          (resid_dropout): Dropout(p=0.1, inplace=False)
        )
        (ln_2): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
        (mlp): GPT2MLP(
          (c_fc): Conv1D(nf=3072, nx=768)
          (c_proj): Conv1D(nf=768, nx=3072)
          (act): NewGELUActivation()
          (dropout): Dropout(p=0.1, inplace=False)
        )
      )
    )
    (ln_f): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
  )
  (lm_head): Linear(in_features=768, out_features=50257, bias=False)
)
```

```
recipe_prompts = [
    'To make butter chicken',
    'For pasta carbonara',
    'To prepare a chocolate cake'
]

print("=== BASELINE RECIPES ===")

baseline2 = {}
for p in recipe_prompts:
    baseline2[p] = generate_text(model2, tokenizer2, p)
    print(f"\nPrompt: {p}")
    print(f"Output: {baseline2[p]}")
```

```
=== BASELINE RECIPES ===

Prompt: To make butter chicken
Output: To make butter chicken, you have to melt the butter first. Then pour some salt in. Butter chicken may nee

Prompt: For pasta carbonara
Output: For pasta carbonara

¼ cup diced onion

½ cup fresh basil finely chopped (or 1/4 cup cumin)

½ cup chopped coriander leaves

½ cup fresh thyme leaves

¼ cup fresh parsley

¼ cup fresh sage leaves

1/4 teaspoon freshly ground black pepper

1/4 teaspoon freshly ground black pepper to taste Instructions In a large skillet over medium heat, melt butter a

Prompt: To prepare a chocolate cake
Output: To prepare a chocolate cake for each participant, roll out the dough and press into a ball of dough. Bake

After the chocolate is done baking, remove the dough from the oven and allow to cool completely. In a separate bo
```

```
recipes = [
    'to make butter chicken start by marinating chicken pieces in yogurt with turmeric chili powder and garam ma
    'heat butter in a pan and fry onions until golden brown then add ginger garlic paste and cook for two minute
    'add tomato puree and cook on low heat for ten minutes until the oil separates from the masala.',
    'add the marinated chicken and cook on medium heat for fifteen minutes until fully cooked.',
    'finish with fresh cream and kasuri methi and serve hot with naan or steamed rice.',
```

```
    'for pasta carbonara boil spaghetti in salted water until al dente and reserve half cup of pasta water.',
    'fry diced pancetta in olive oil until crispy and set aside.',
    'whisk together egg yolks parmesan cheese and black pepper in a bowl.',
    'toss the hot pasta with pancetta and remove from heat then quickly stir in the egg mixture.',
    'the residual heat will cook the eggs into a creamy sauce and serve immediately with extra parmesan.',
    'to prepare vegetable stir fry heat sesame oil in a wok on high heat.',
    'add sliced bell peppers broccoli florets and snap peas and toss for three minutes.',
    'pour in soy sauce oyster sauce and a pinch of sugar and stir well.',
    'add minced garlic and ginger and cook for one more minute until fragrant.',
    'serve the stir fry over steamed jasmine rice and garnish with sesame seeds.',
    'for chocolate chip cookies cream together butter and sugar until light and fluffy.',
    'beat in eggs one at a time then add vanilla extract and mix well.',
    'fold in flour baking soda and salt then gently stir in chocolate chips.',
    'scoop dough onto a baking sheet and bake at 180 degrees for twelve minutes until golden.',
    'let cookies cool on the tray for five minutes before transferring to a wire rack.'
]

dataset2 = Dataset.from_dict({'text': recipes})
```

```
tok2 = dataset2.map(
    lambda x: tokenizer2(x['text'], truncation=True, padding='max_length', max_length=128),
    batched=True,
    remove_columns=['text']
)

split2 = tok2.train_test_split(test_size=0.15, seed=42)
```

```
Map:   0%|          | 0/20 [00:00<?, ? examples/s]
```

```
collator2 = DataCollatorForLanguageModeling(tokenizer=tokenizer2, mlm=False)

args2 = TrainingArguments(
    output_dir='./gpt2-recipes',
    num_train_epochs=10,
    per_device_train_batch_size=4,
    learning_rate=5e-5,
    logging_steps=10,
    fp16=torch.cuda.is_available()
)
trainer2 = Trainer(
    model=model2,
    args=args2,
    train_dataset=split2['train'],
    eval_dataset=split2['test'],
    data_collator=collator2
)

trainer2.train()
```

[30/30 00:07, Epoch 10/10]

Step	Training Loss
10	3.855437
20	2.753023
30	2.566286

```
Writing model shards:   0%|          | 0/1 [00:00<?, ?it/s]
TrainOutput(global_step=30, training_loss=3.058248456319173, metrics={'train_runtime': 8.1974,
'train_samples_per_second': 20.738, 'train_steps_per_second': 3.66, 'total_flos': 11104911360000.0,
'train_loss': 3.058248456319173, 'epoch': 10.0})
```

```
eval2 = trainer2.evaluate()
print("Perplexity:", math.exp(eval2["eval_loss"]))

print("\n=== FINE-TUNED RECIPES ===")

for p in recipe_prompts:
    ft = generate_text(model2, tokenizer2, p)
    print(f"\nPrompt: {p}")
    print(f"Baseline:   {baseline2[p][:120]}")
    print(f"Fine-Tuned: {ft[:120]}")
```

[1/1 : < :]

```
Perplexity: 18.55948851922575

=== FINE-TUNED RECIPES ===

Prompt: To make butter chicken
Baseline:   To make butter chicken, you have to melt the butter first. Then pour some salt in. Butter chicken may
Fine-Tuned: To make butter chicken stir fry chicken in egg and cook until lightly golden. Saute chicken for two m

Prompt: For pasta carbonara
Baseline:   For pasta carbonara

¼ cup diced onion

½ cup fresh basil finely chopped (or 1/4 cup cumin)
```